

ATmega328P Register Descriptions For the Sleep-Mode Task

Register Configuration

The sleep and wake-up behavior of the ATmega328P is controlled through several microcontroller registers. The configuration used in this project is described below.

Sleep Mode Control Register (SMCR)

Bit	7	6	5	4	3	2	1	0	
0x33 (0x53)	–	–	–	–	SM2	SM1	SM0	SE	SMCR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 37

Purpose: Controls MCU sleep mode and sleep enable functionality.

Bit	Name	Setting	Purpose
[2:1]	SM1:SM0	10	Select Power-down sleep mode (CPU + clocks stopped, only external interrupts can wake)
[0]	SE	1	Sleep Enable - Allows sleep instruction to activate sleep mode
[0]	SE	0	Cleared after wake-up to disable sleep

```
SMCR = (1 << SE) | (1 << SM1); // Enable sleep + Power-down mode (SM0 left as 0)
SMCR &= ~(1 << SE);           // Clear SE after wake-up
```

External Interrupt Mask Register (EIMSK)

Datasheet page: PAGE 55

Purpose: Enables/disables external interrupts.

Bit	7	6	5	4	3	2	1	0	
0x1D (0x3D)	–	–	–	–	–	–	INT1	INT0	EIMSK
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	Name	Setting	Purpose
[0]	INT0	1	Enable INT0 interrupt (on PD2) - Critical for wake-up from sleep

```
EIMSK |= (1 << INT0); // Enable INT0 to wake MCU from sleep
```

External Interrupt Control Register (EICRA)

Bit	7	6	5	4	3	2	1	0	
(0x69)	–	–	–	–	ISC11	ISC10	ISC01	ISC00	EICRA
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bits	Name	Setting	Meaning
[1:0]	ISC01:ISC00	00	Low-level trigger - Interrupt fires when INT0 pin is held LOW

```
EICRA &= ~((1 << ISC01) | (1 << ISC00)); // Clear both bits = low-level trigger
```

Why Low-Level Trigger?

- The button pulls PD2 LOW when pressed (internal pull-up keeps it HIGH normally).
- Low-level trigger detects and maintains interrupt as long as button is held.

External Interrupt Flag Register (EIFR)

Bit	7	6	5	4	3	2	1	0	
0x1C (0x3C)	–	–	–	–	–	–	INTF1	INTF0	EIFR
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 55

Purpose: Flag that indicates an external interrupt has occurred (Not explicitly modified, but important for understanding ISR flow).

Bit	Name	Auto-Behavior	Manual Clear
[0]	INTF0	Set by hardware when INT0 is triggered	Writing 1 to bit clears it (auto-cleared by ISR execution)

Status Register (SREG)

Bit	7	6	5	4	3	2	1	0	
0x3F (0x5F)	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 11

Purpose: Global interrupt enable/disable control.

Bit	Name	Setting	Purpose
[7]	I (Global Interrupt Enable)	1 (via sei())	Enable all interrupts - Essential for INT0 to wake MCU

```
sei(); // Set Interrupt Enable = macro that sets SREG[7]
```

Critical Point: Without sei(), the INT0 interrupt cannot execute, preventing wake-up!

General I/O Registers

DDRD (Data Direction Register D) - 0x2A

Bit	7	6	5	4	3	2	1	0	
0x0A (0x2A)	DDD7	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0	DDRD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 73

Purpose: Configure port D pins as input or output.

Bit	Pin	Setting	Purpose
[2]	DDD2 (PD2)	0	Configure PD2 as INPUT for button/INT0

```
DDRD &= ~(1 << DDD2); // Clear bit = input
```

PORTD (Port D Data Register) - 0x2B

Bit	7	6	5	4	3	2	1	0	
0x0B (0x2B)	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0	PORTD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 73

Purpose: Output logic levels; also enables pull-up resistors when pin is input.

Bit	Pin	Setting	Purpose
[2]	PORTD2 (PD2)	1	Enable internal pull-up resistor (PD2 is input, so this activates pull-up)

```
PORTD |= (1 << PORTD2); // Set bit = pull-up on input pin
```

Why Pull-Up? Button pulls to GND (active LOW), pull-up keeps pin HIGH at rest.

DDRB (Data Direction Register B) - 0x24

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 72

Purpose: Configure port B pins as input or output.

Bit	Pin	Setting	Purpose
[0]	DDB0 (PB0)	1	Configure PB0 as OUTPUT for LED
[1]	DDB1 (PB1)	1	Configure PB1 as OUTPUT for BUZZER

```
DDRB |= (1 << PB0) | (1 << PB1); // Set bit = output
```

PORTB (Port B Data Register) - 0x25

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 72

Purpose: Output logic levels to port pins.

Bit	Pin	Setting	Purpose
[0]	PORTB0 (PB0)	1	Turn LED ON
[0]	PORTB0 (PB0)	0	Turn LED OFF
[1]	PORTB1 (PB1)	1	Turn BUZZER ON
[1]	PORTB1 (PB1)	0	Turn BUZZER OFF

```
PORTB |= (1 << PB0) | (1 << PB1); // Set bit = LED on
PORTB &= ~(1 << PB0) | (1 << PB1)); // Clear bit = LED off
```

ADCSRA (ADC Control & Status Register A) - 0x7A

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 218

Purpose: Controls ADC (Analog-to-Digital Converter).

Bit	Name	Setting	Purpose
[7]	ADEN	0	Disable ADC to reduce power consumption in sleep mode

```
ADCSRA &= ~(1 << ADEN); // Disable ADC before sleep
```

Why Disable? ADC consumes power; disable it before entering Power-down sleep mode.

Part 2: Timer Registers (lcd_test.c - Delay Functions)

TCCR0A (Timer/Counter 0 Control Register A) - 0x44

Bit	7	6	5	4	3	2	1	0	
0x24 (0x44)	COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
Read/Write	R/W	R/W	R/W	R/W	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 84

Purpose: Configure Timer0 mode, waveform generation, and output compare.

Bit	Setting	Purpose
All bits	0x00	Normal mode (simple counter, no PWM)

```
TCCR0A = 0x00; // Reset to normal mode
```

TCCR0B (Timer/Counter 0 Control Register B) - 0x45

Bit	7	6	5	4	3	2	1	0	
0x25 (0x45)	FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
Read/Write	W	W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 86

Purpose: Start/stop timer and set prescaler value.

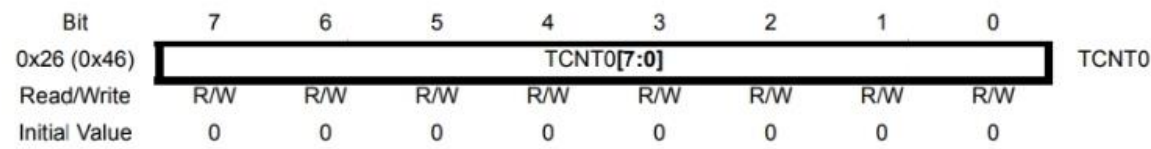
Bits	Name	Setting	Purpose
[2:0]	CS02:CS00	011 (CS01=1, CS00=1)	Prescaler 64 - Clock = 16MHz/64 = 250kHz
All	-	0x00	Stop timer

```
TCCR0B |= (1 << CS00) | (1 << CS01); // Start with /64 prescaler
TCCR0B = 0x00; // Stop timer
```

Timing Calculation:

- Timer clock: 250kHz
- 1 timer tick: 1 / 250kHz = 4µs
- Max count 256 cycles = 1024µs = ~1ms (for delay_ms)

TCNT0 (Timer/Counter 0 Value Register) - 0x46



Datasheet page: PAGE 87

Purpose: Timer counter value (read/write).

```
TCNT0 = 256 - 250; // Preload for 250 cycles = 1ms delay
TCNT0 = 256 - ticks; // Preload for microsecond delays
```

How it works:

- Preload value, counter counts UP.
- When it overflows (256 → 0), TOV0 flag is set.
- Time to overflow = (256 - preload) × clock period.

TIFR0 (Timer/Counter 0 Interrupt Flag Register) - 0x35

Datasheet page: PAGE 88

Purpose: Timer interrupt flags (overflow, compare, etc.).

Bit	Name	Setting	Purpose
[0]	TOV0	Write 1	Clear overflow flag (write 1 to clear)
[0]	TOV0	Read until 1	Wait for overflow - Flag set by hardware when timer rolls over

```
TIFR0 |= (1 << TOV0); // Clear flag
while (!(TIFR0 & (1 << TOV0))); // Wait for overflow
```

Part 3: I2C/TWI Registers (lcd_test.c - I2C Functions)

TWBR (TWI Bit Rate Register) - 0xB8

Bit	7	6	5	4	3	2	1	0	
(0xB8)	TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0	TWBR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Datasheet page: PAGE 198

Purpose: Set I2C clock frequency.

Setting	Value	Clock Frequency
TWBR = 72	100 kHz (at 16MHz CPU)	Standard I2C speed

Formula: $SCL\ Freq = CPU_Freq / (16 + 2 \times TWBR \times Prescaler)$

```
TWBR = 72; // ~100kHz I2C clock
```

TWSR (TWI Status Register) - 0xB9

Bit	7	6	5	4	3	2	1	0	
(0xB9)	TWS7	TWS6	TWS5	TWS4	TWS3	–	TWPS1	TWPS0	TWSR
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	1	1	1	1	1	0	0	0	

Datasheet page: PAGE 200

Purpose: I2C status and prescaler control.

Bits	Name	Setting	Purpose
[1:0]	TWPS	00	Prescaler 1:1 (no prescaling)

```
TWSR = 0x00; // Set prescaler to 1:1
```

TWCR (TWI Control Register) - 0xBC

Bit	7	6	5	4	3	2	1	0	
(0xBC)	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE	TWCR
Read/Write	R/W	R/W	R/W	R/W	R	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

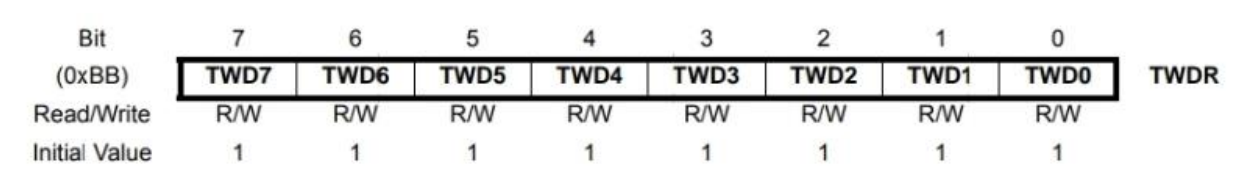
Datasheet page: PAGE 199

Purpose: Control I2C start/stop conditions and data transmission.

Bit	Name	Setting	Purpose
[7]	TWINT	1	Write to clear flag after each I2C operation
[6]	TWEA	0	Acknowledge control (not used in master-only)
[5]	TWSTA	1	Generate START condition
[4]	TWSTO	1	Generate STOP condition
[2]	TWEN	1	Enable TWI peripheral

```
TWCR = (1 << TWEN);                // Enable I2C (line 119)
TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN); // START (line 133)
TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN); // STOP (line 150)
TWCR = (1 << TWINT) | (1 << TWEN);           // Send data (line 170)
```

TWDR (TWI Data Register) - 0xBB



Datasheet page: PAGE 200

Purpose: I2C data byte to transmit or receive.

```
TWDR = data;           // Load byte to send on I2C bus
I2C_write(PCF8574_ADDR << 1); // Send slave address (0x20 << 1 = 0x40)
I2C_write(data);       // Send data byte to PCF8574
```

I2C Transaction Flow:

- 1. START condition (TWSTA)
- 2. Send address + write bit (0x40)
- 3. Send data byte (LCD control bits)
- 4. STOP condition (TWSTO)

Summary Table: All Registers Used

Register	Location	Primary Purpose	Key Bits
SMCR	0x53	Sleep mode control	SE, SM1:SM0
EIMSK	0x1D	External interrupt enable	INT0
EICRA	0x69	External interrupt trigger mode	ISC01:ISC00
EIFR	0x1C	External interrupt flag	INTF0

SREG	0x5F	Status register (global interrupts)	I (bit 7)
DDRD	0x2A	Port D direction	DDD2
PORTD	0x2B	Port D data/pull-up	PORTD2
DDRB	0x24	Port B direction	DDB0
PORTB	0x25	Port B data	PORTB0
ADCSRA	0x7A	ADC control	ADEN
TCCR0A	0x44	Timer0 mode	(all 0 = normal)
TCCR0B	0x45	Timer0 prescaler	CS02:CS00
TCNT0	0x46	Timer0 counter value	(0-255)
TIFR0	0x35	Timer0 flags	TOV0
TWBR	0xB8	I2C bit rate	72 = 100kHz
TWSR	0xB9	I2C status/prescaler	TWPS
TWCR	0xBC	I2C control	TWINT, TWSTA, TWSTO, TWEN
TWDR	0xBB	I2C data	(8-bit data byte)

REFERENCE

- Microchip Technology. (2015). *ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash Datasheet* (Rev. 7810D–AVR–01/15). Microchip Technology Inc. [Official Microchip Datasheet](#). [1]